OPERATING SYSTEMS

UE25CS242A



Memory Management

Pavan A C
Neha Sharma

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

PES UNIVERSITY
CELEBRATING 50 YEARS

NUMA (Non-Uniform Memory Access) Basics and Memory Locality, Clock Page Replacement.

Pavan A C

Neha Sharma

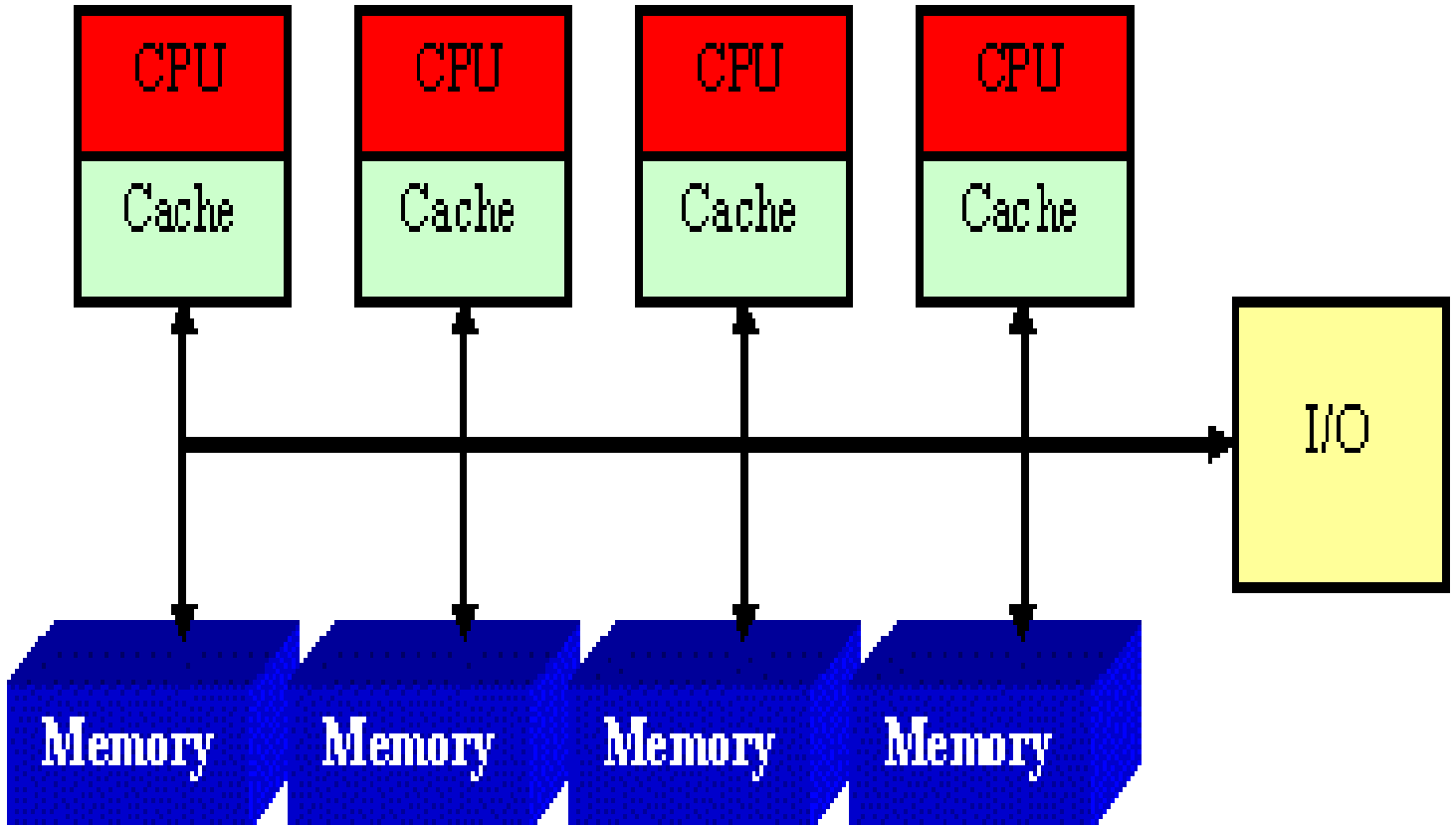
Department of Computer Science & Engineering

Non-uniform Memory Access

PES UNIVERSITY
CELEBRATING 50 YEARS

- In systems with multiple CPUs, a given CPU can access some sections of main memory faster than it can access others.
- These performance differences are caused by how CPUs and memory are interconnected in the system.
- Systems in which memory access times vary significantly are known collectively as **non-uniform memory access (NUMA)** systems, and without exception, they are slower than systems in which memory and CPUs are located on the same motherboard
- Managing which page frames are stored at which locations can significantly affect performance in NUMA systems. If we treat memory as uniform in such a system, CPUs may wait significantly longer for memory access than if we modify memory allocation algorithms to take NUMA into account.

NUMA Architecture



Components of a NUMA System

- **Processors (CPUs):** Execute processes and access both local and remote memory.
- **Local Memory:** Memory directly connected to a processor with the lowest access latency.
- **Remote Memory:** Memory connected to another processor; accessed through the interconnect.
- **High-Speed Interconnect:** Transfers data between processors and memory nodes.
- **Memory Controller:** Handles memory requests and manages data transfer.
- **Operating System:** Schedules processes and allocates memory to improve memory locality.

Memory locality

Memory locality refers to the tendency of a program to access the same memory locations or nearby memory locations repeatedly over a short period of time. The principle of locality is the basis for efficient memory hierarchy design (registers, cache, RAM and virtual memory). Memory locality improves system performance by:

- Reducing memory access time.
- Increasing CPU cache hit rate.
- Decreasing cache misses.
- Improving virtual memory and paging efficiency.
- Enhancing overall execution speed.

Types of Memory locality

1. Temporal Locality (Locality in Time)

If a memory location is accessed, it is likely to be accessed again in the near future.

Applications:

CPU cache stores recently accessed data.

Frequently used instructions remain in the instruction cache.

2. Spatial Locality (Locality in Space)

If a memory location is accessed, nearby memory locations are likely to be accessed soon.

Applications:

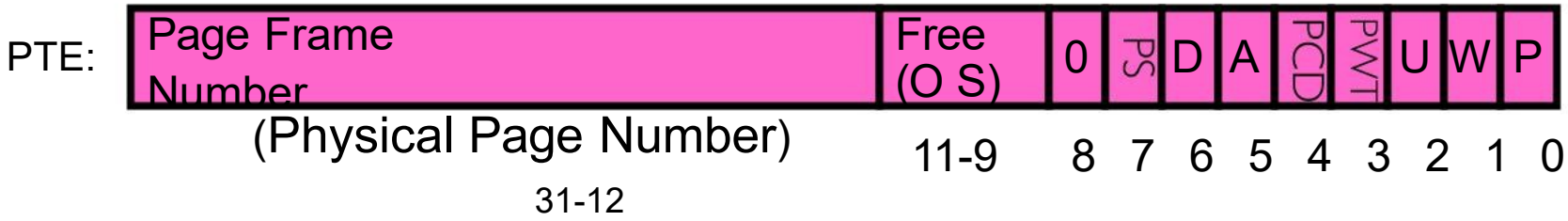
Cache loads an entire cache block (cache line), not just one memory location.

Sequential file reading benefits from spatial locality.

Approximating LRU: PTE bits

- Which bits of a PTE entry can help us approximate LRU?

Intel PTE:



– The "Present" bit (called "Valid" elsewhere):

» P==0: Page is invalid and a reference will cause page fault

» P==1: Page frame number is valid and MMU is allowed to proceed with translation

The "Accessed" bit (called "Use" elsewhere):

» A==0: Page has not been accessed (or used) since last time software set A→0

» A==1: Page has been accessed (or used) since last time software set A→0

Approximating LRU: PTE bits contd…

– The “Writable” bit (could have opposite sense and be called “Read-only”):
  » W==0: Page is read-only and cannot be written.
  » W==1: Page can be written

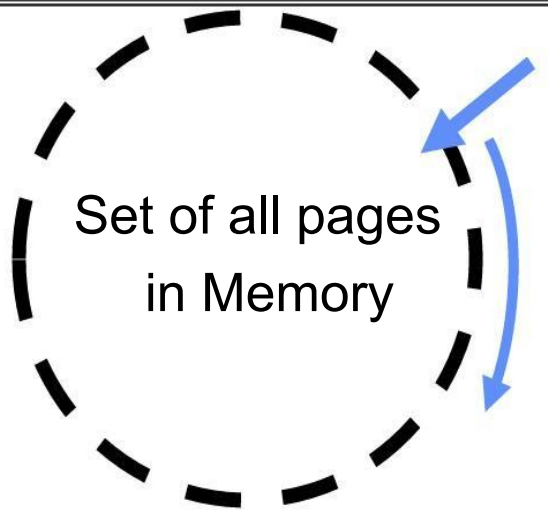
– The “Dirty” bit (called “Modified” elsewhere):

» D==0: Page has not been modified (written) since PTE was loaded
» D==1: Page has changed since PTE was loaded

PES UNIVERSITY
CELEBRATING 50 YEARS

Approximating LRU: Clock Algorithm





Single Clock Hand:

Advances only on page fault

Check for pages not used recently

Mark pages as not used recently

- Clock Algorithm: Arrange physical pages in circle with single clock hand
  - Approximate LRU
  - Replace an old page, not the oldest page

Approximating LRU: Clock Algorithm

PES UNIVERSITY
CELEBRATING 50 YEARS

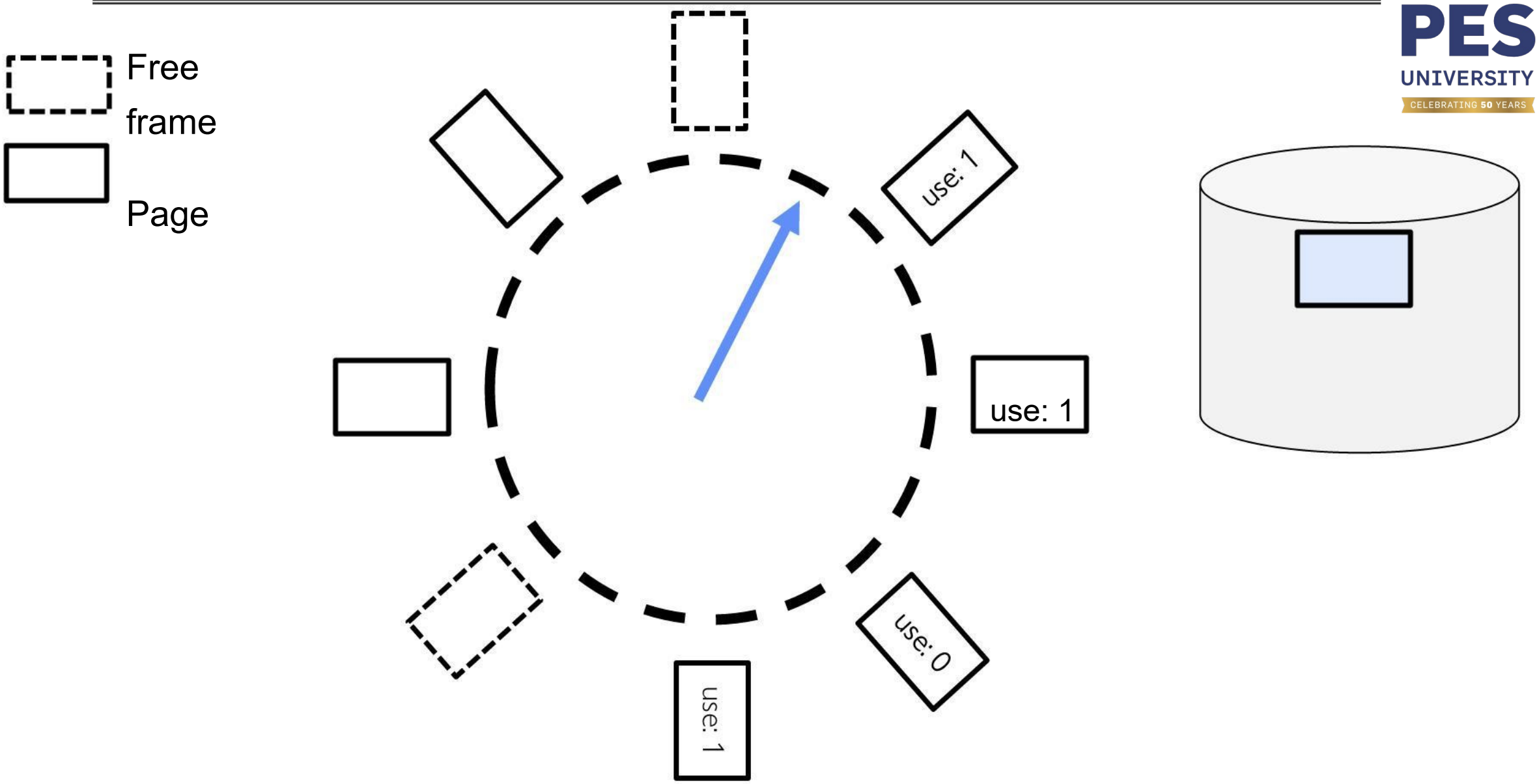
- Details:

– Hardware “use” bit per physical page (called “accessed” in Intel architecture):
  » Hardware sets use bit on each reference
  » If use bit isn’t set, means not referenced in a long time
– On page fault:
  » Advance clock hand (not real time)
  » Check use bit:

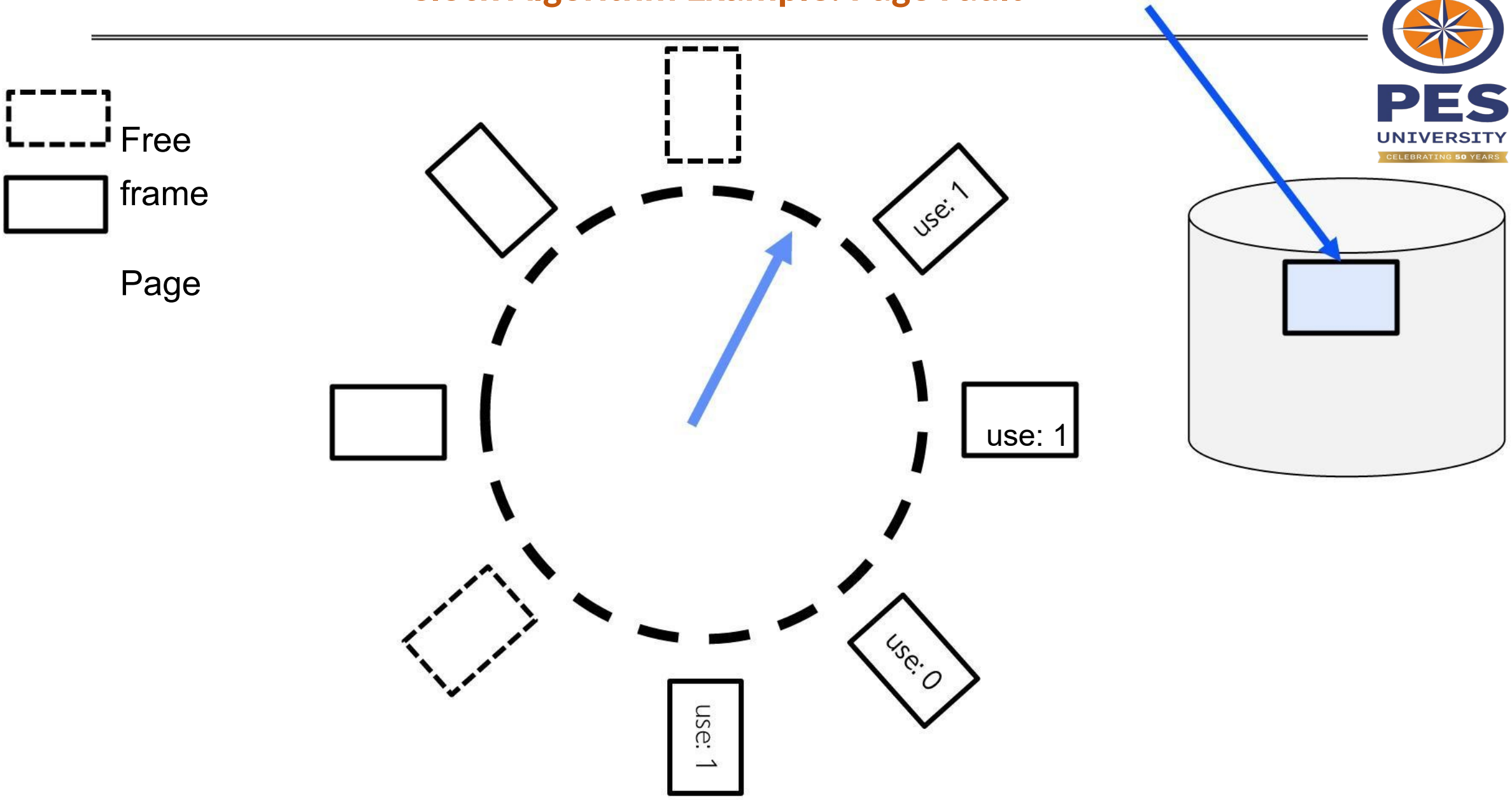
    1→ used recently; clear and leave alone

    0→ selected candidate for replacement

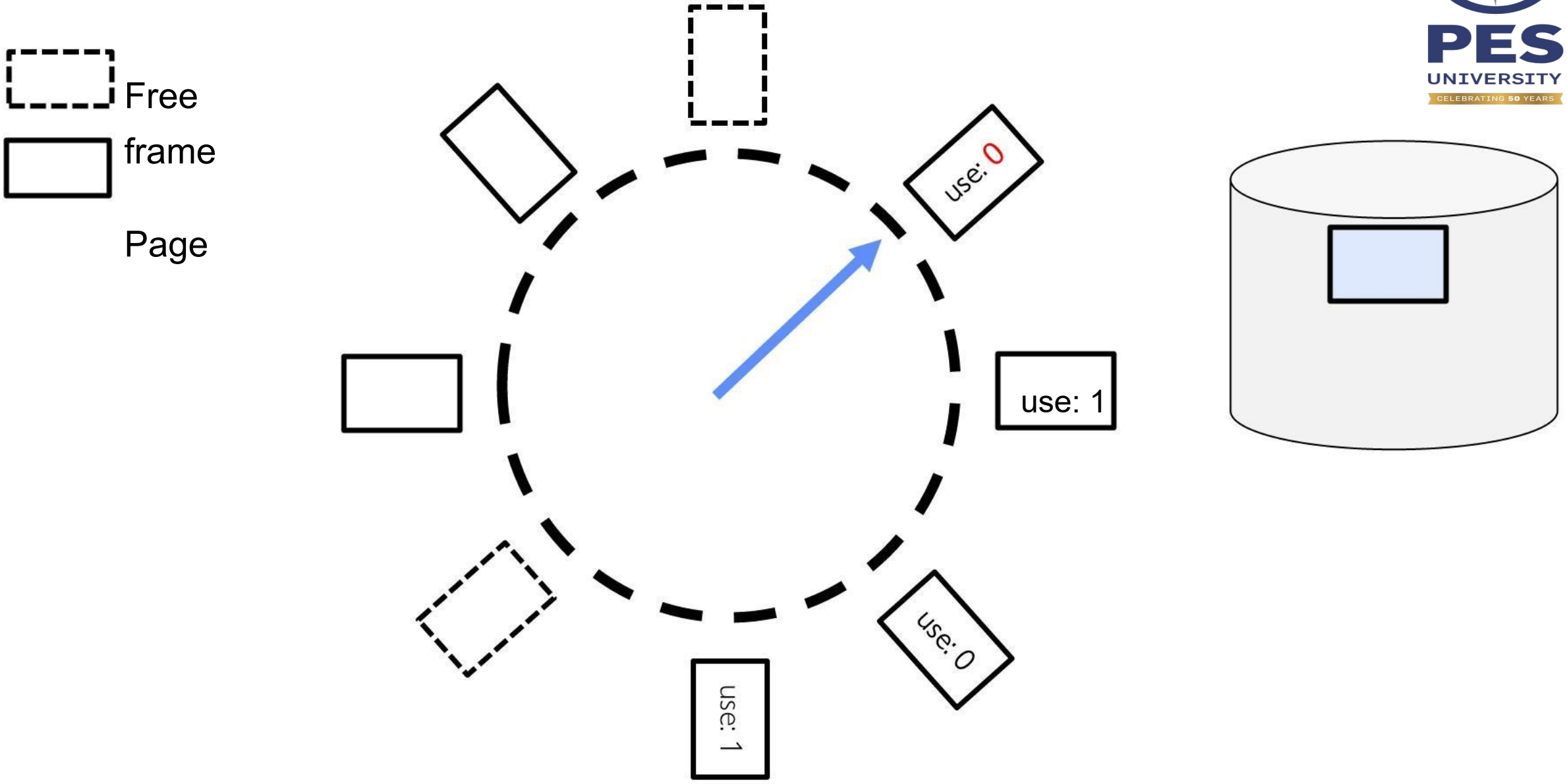
Clock Algorithm Example



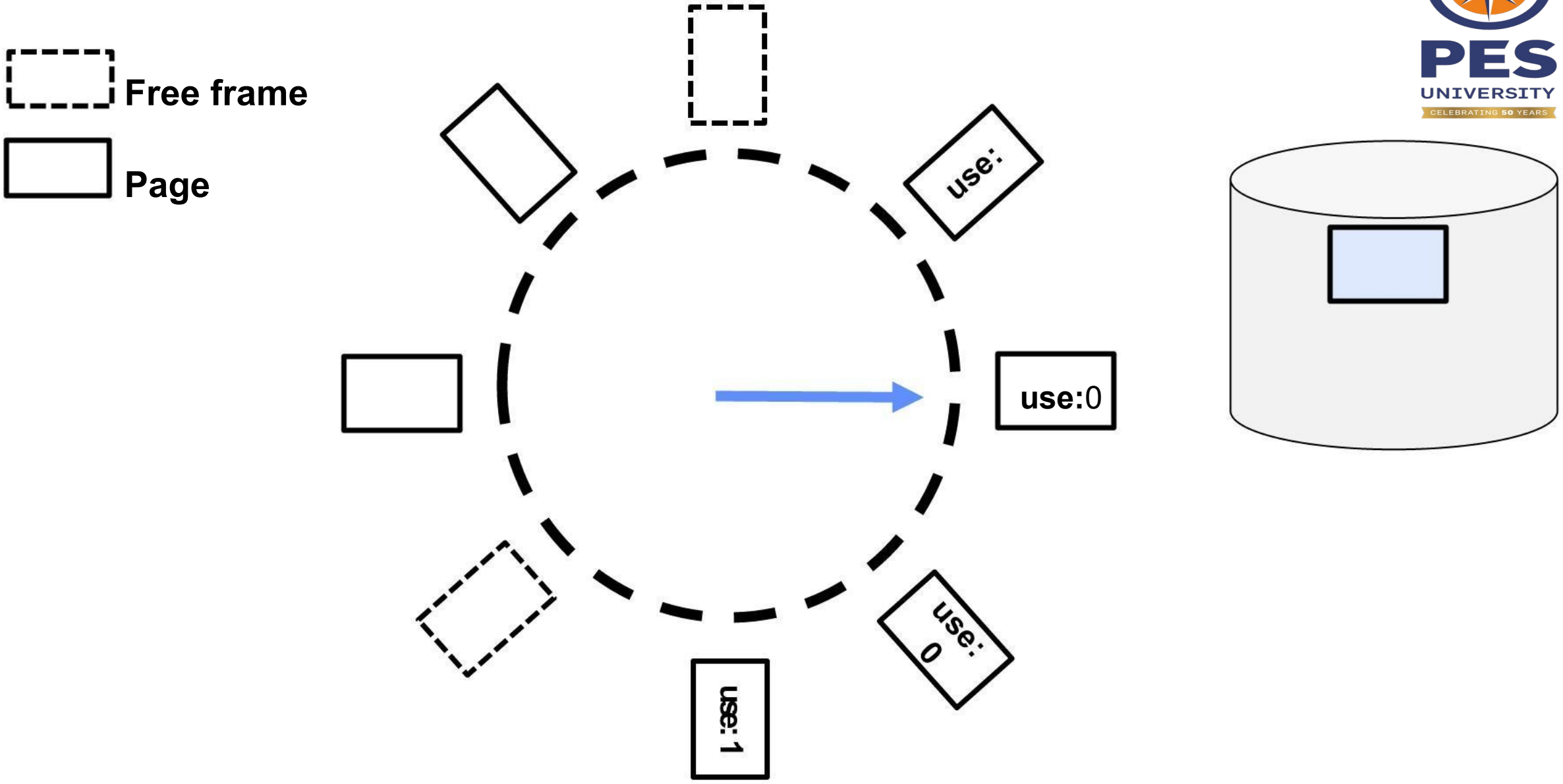
Clock Algorithm Example: Page Fault



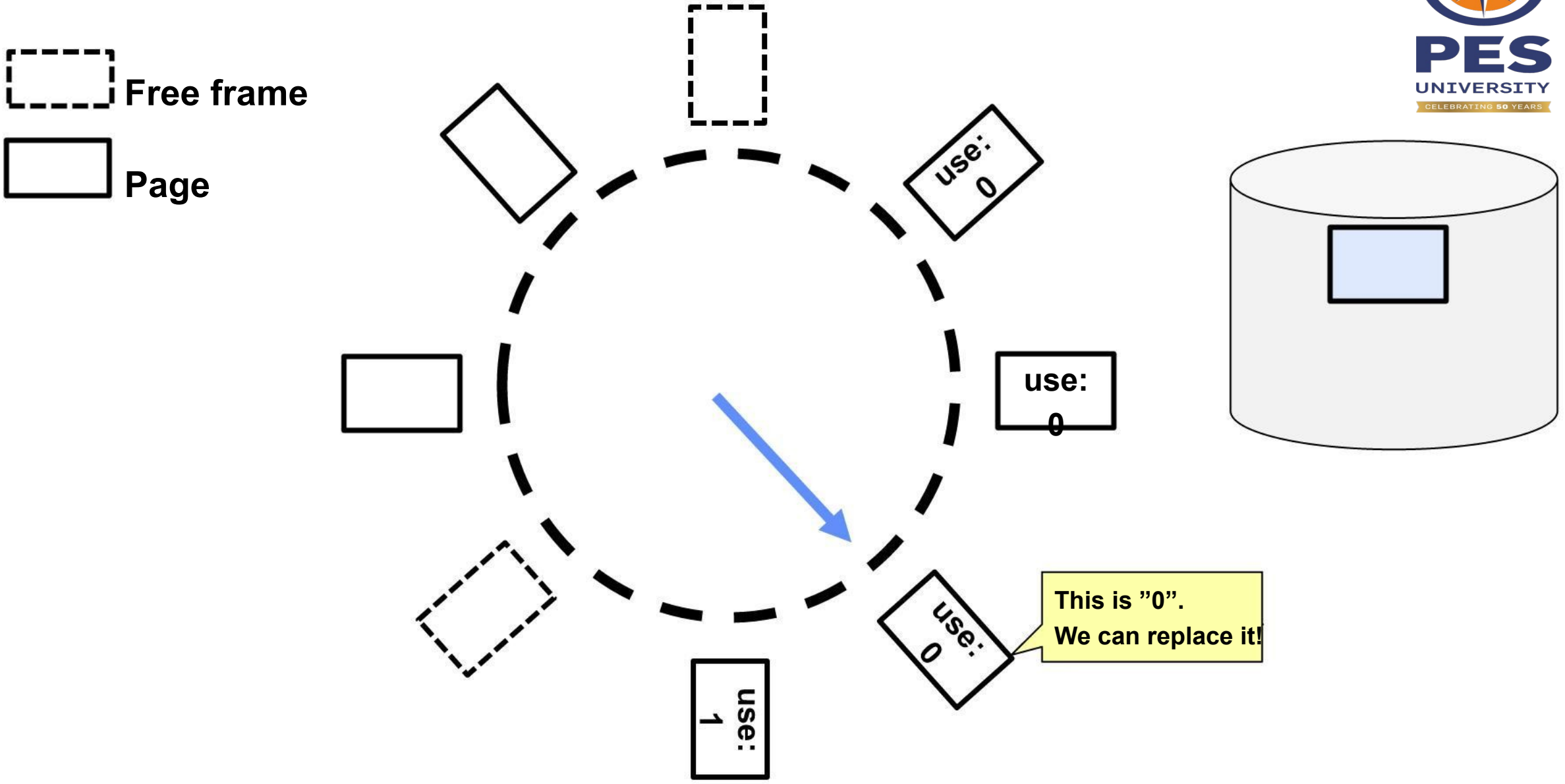
Clock Algorithm Example: Page Fault



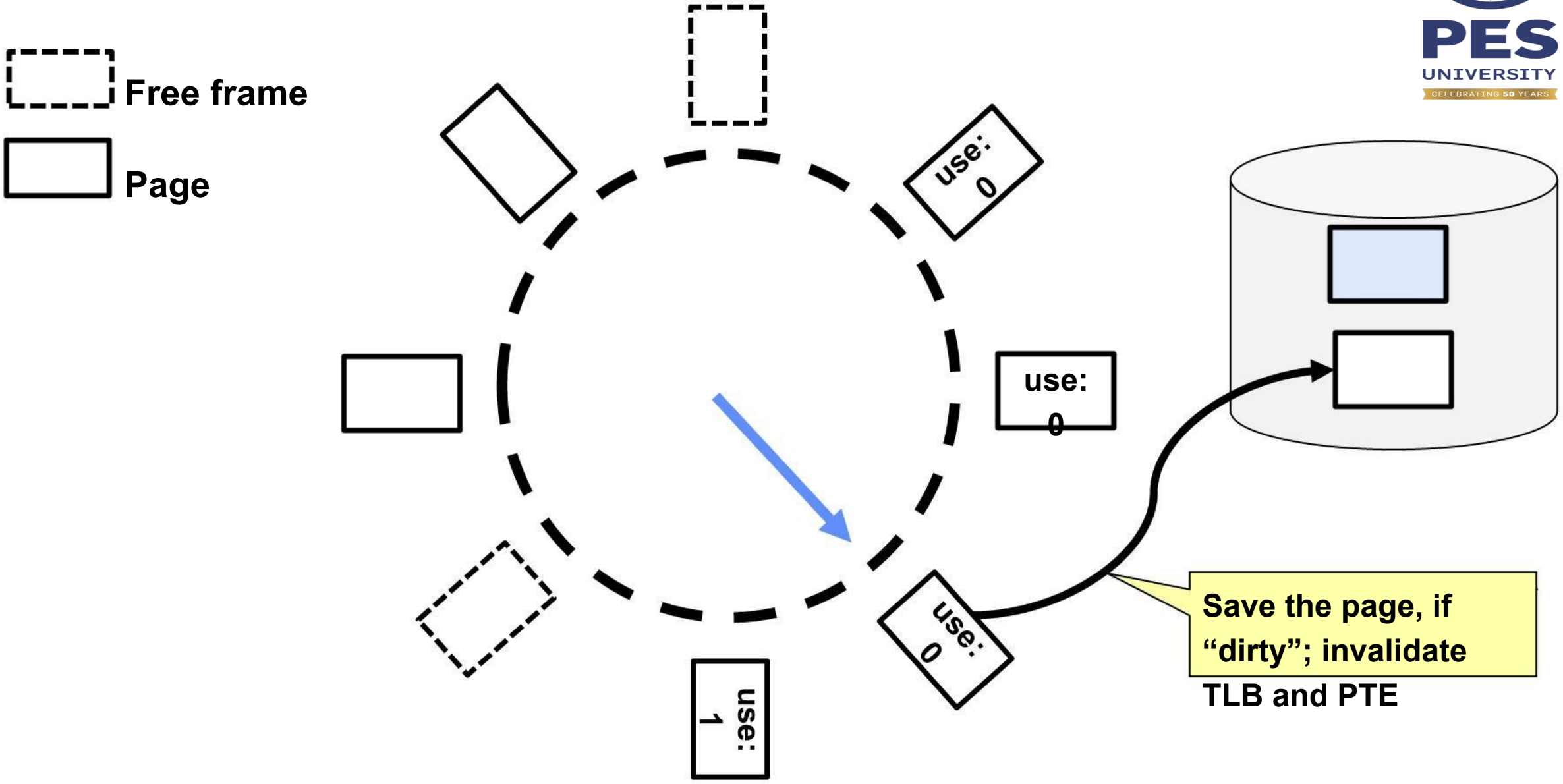
Clock Algorithm Example: Page Fault



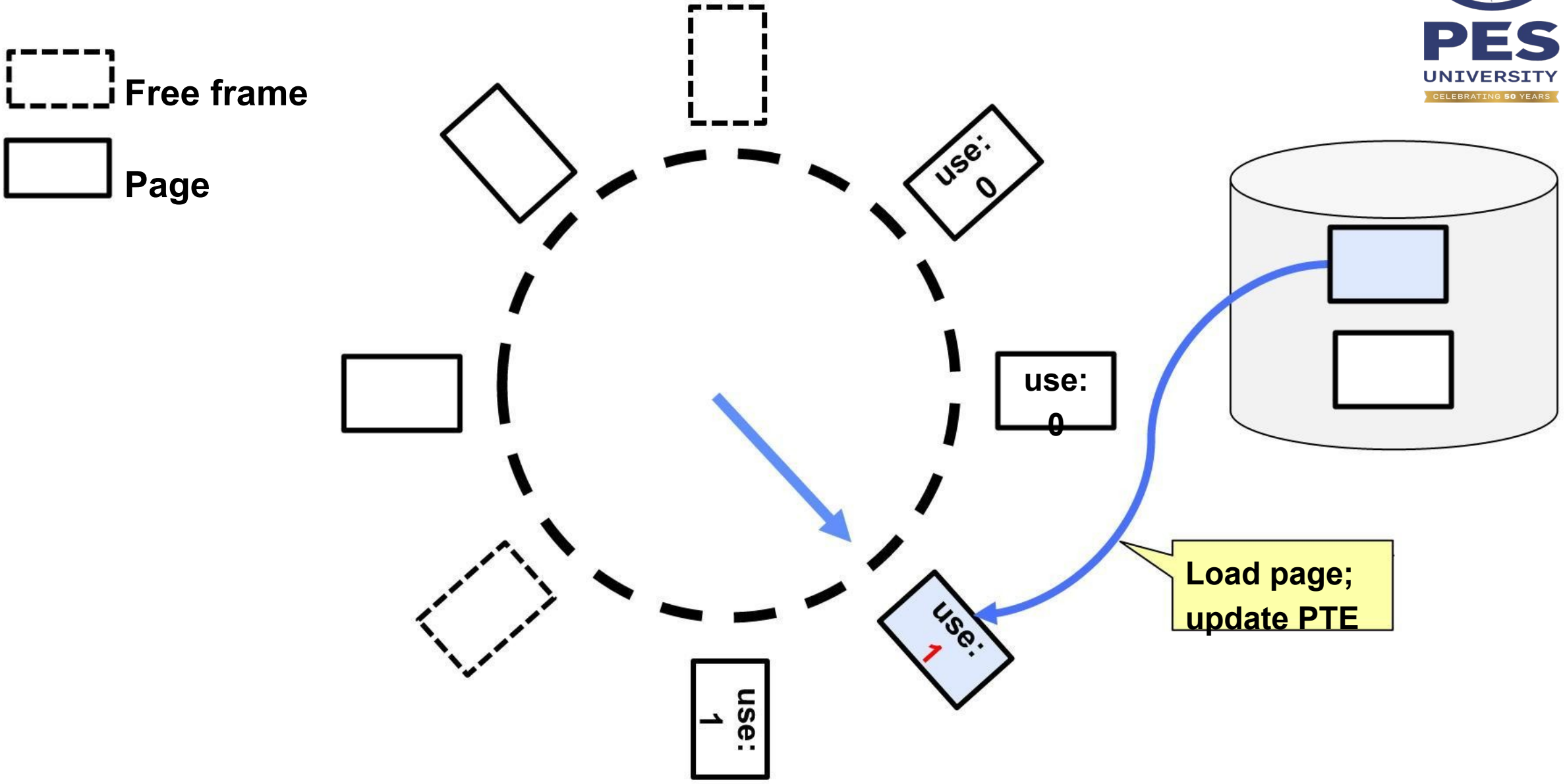
Clock Algorithm Example: Page Fault



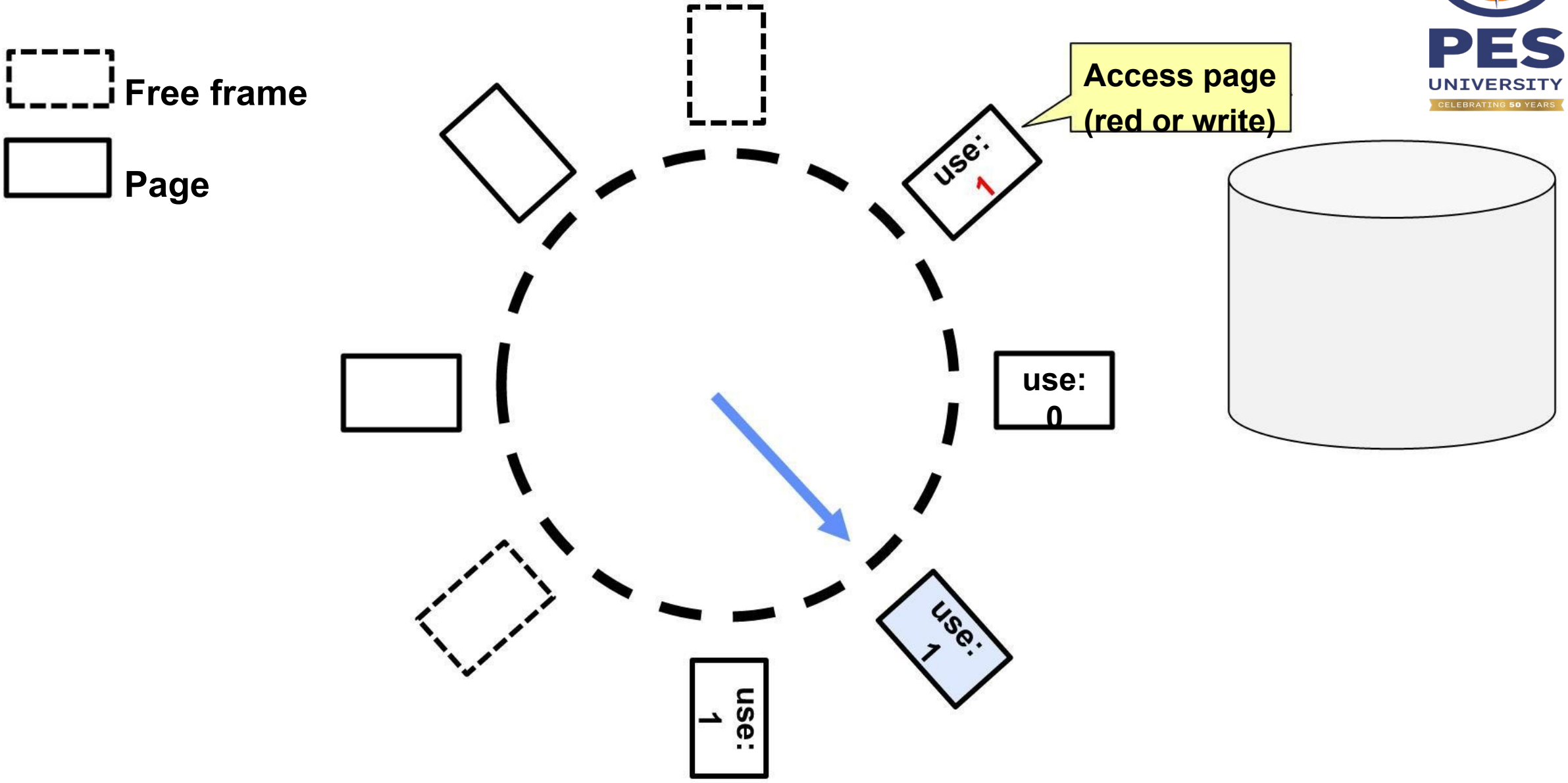
Clock Algorithm Example: Page Fault



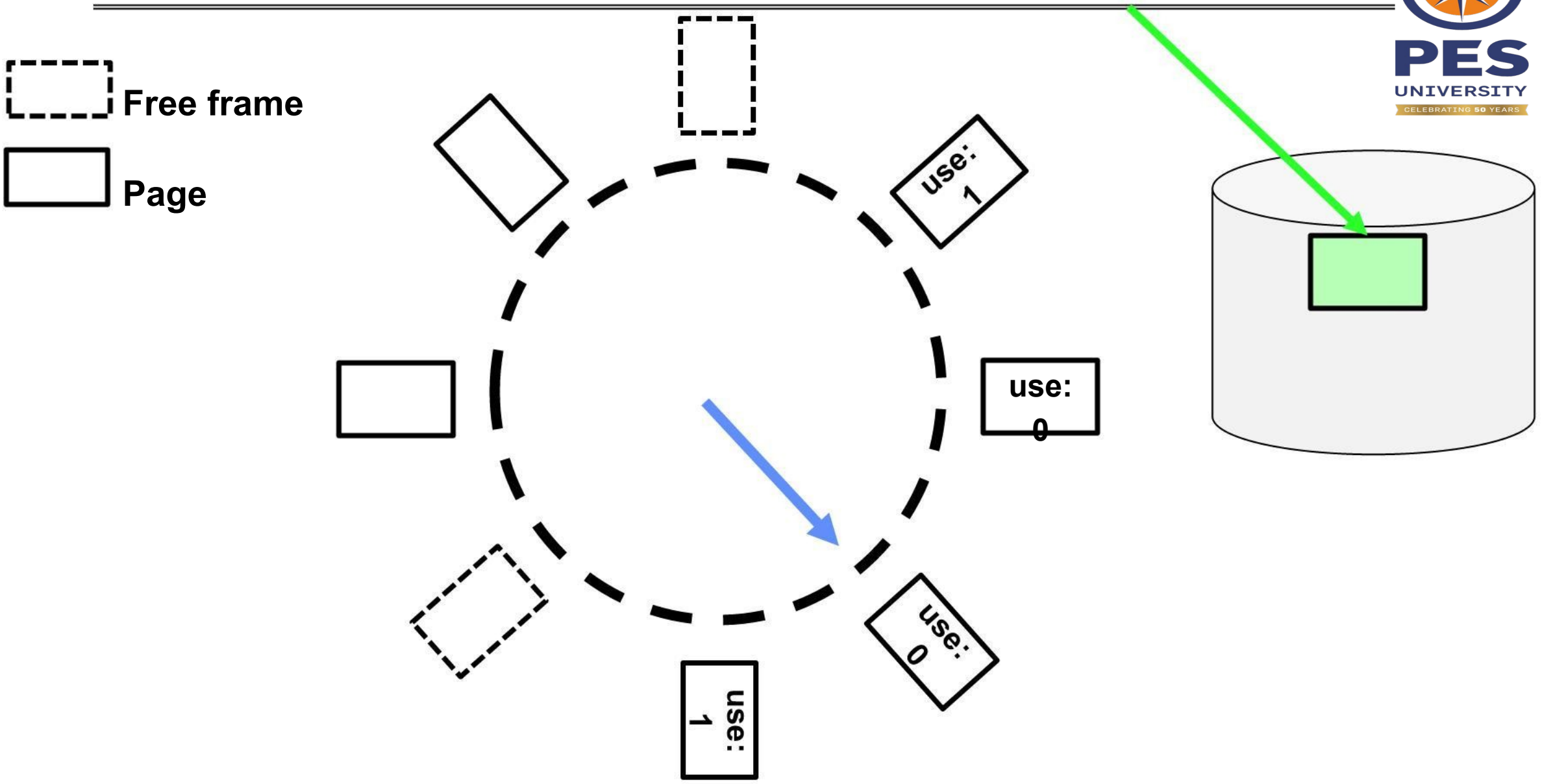
Clock Algorithm Example: Page Fault



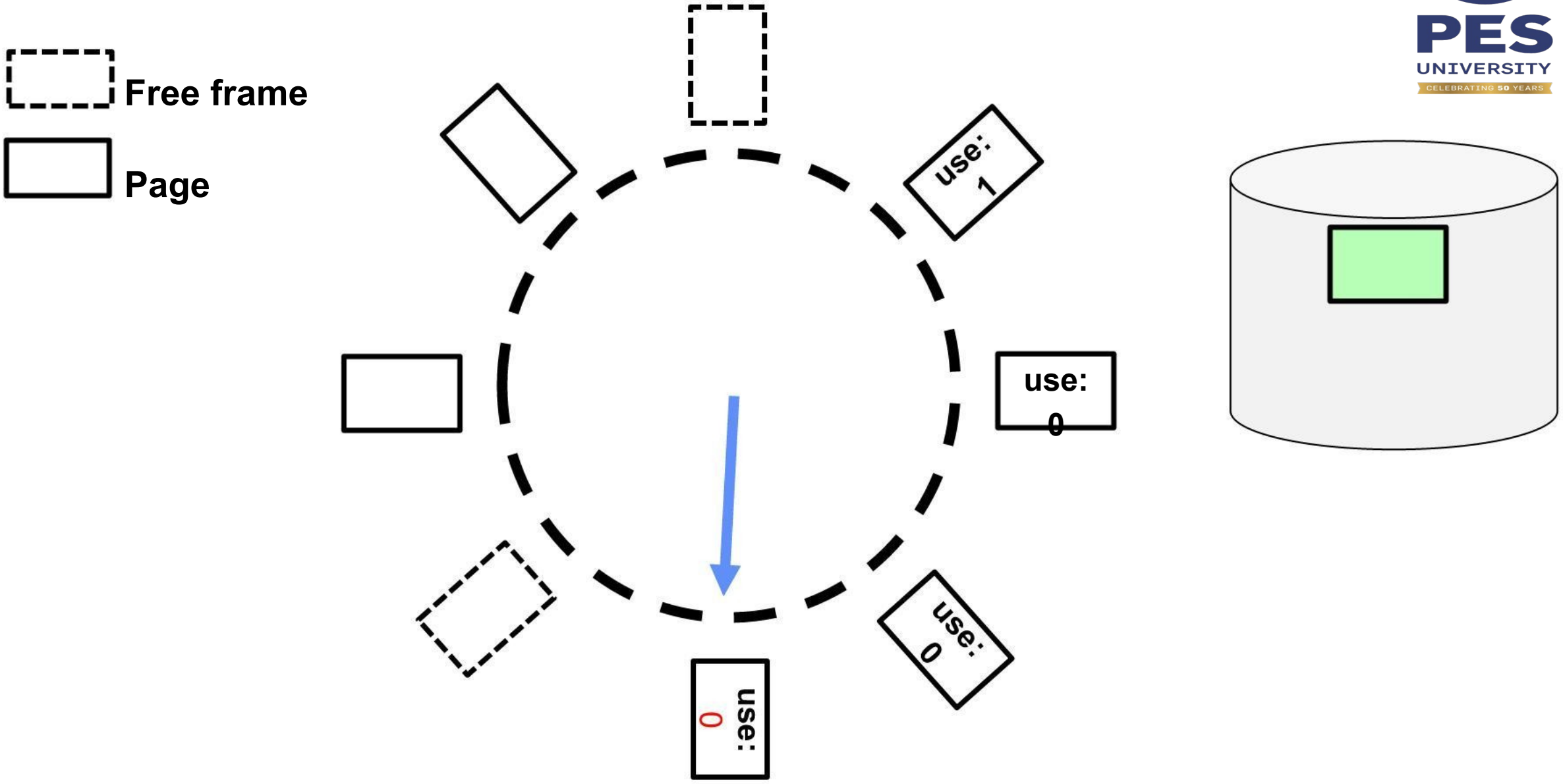
Clock Algorithm Example



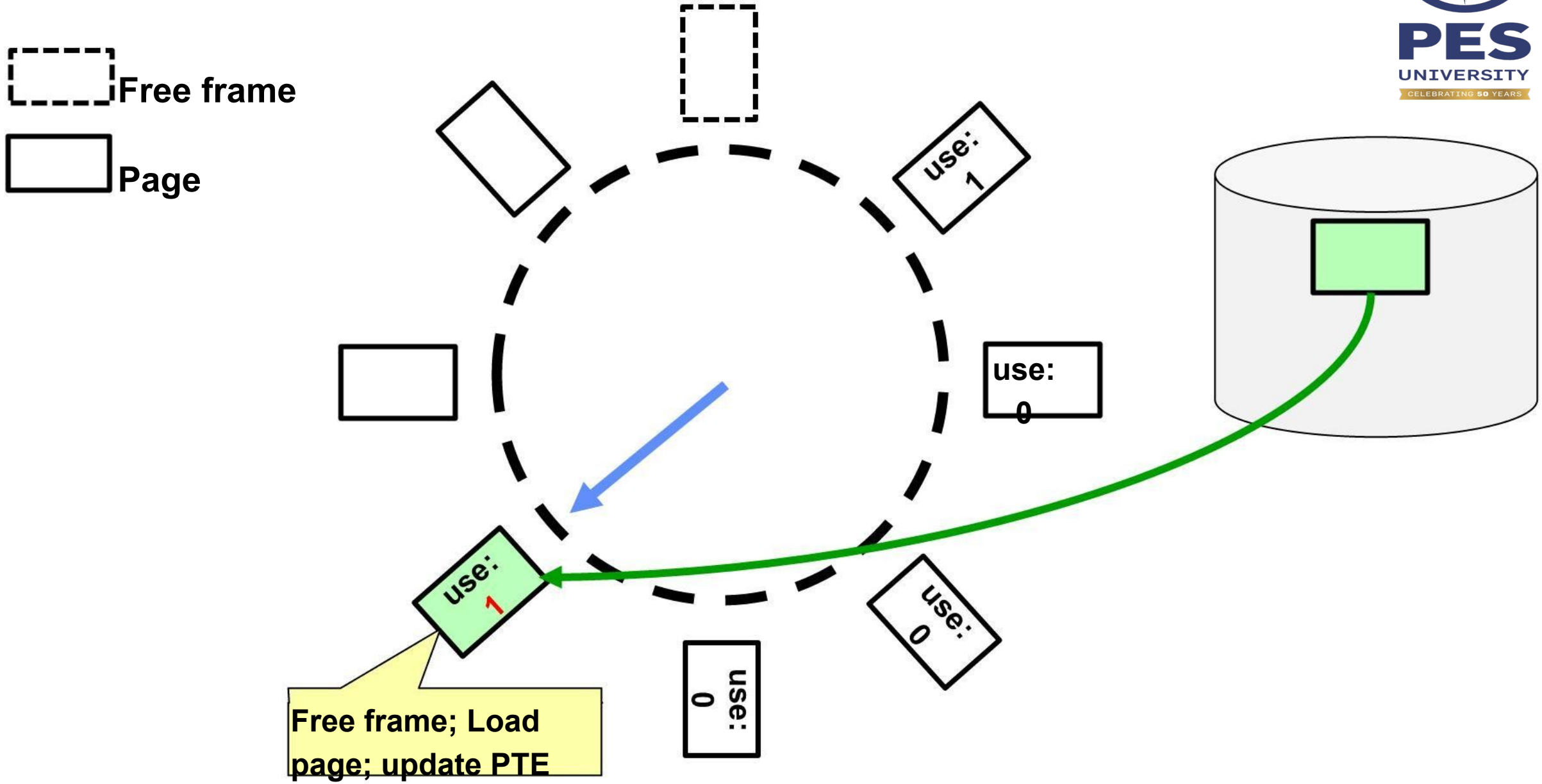
Clock Algorithm Example: Another Page Fault



Clock Algorithm Example: Another Page Fault

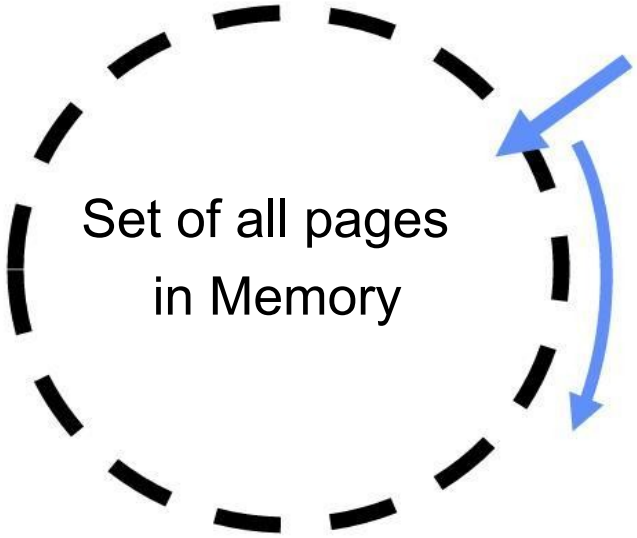


Clock Algorithm Example: Another Page Fault



Clock Algorithm: More details

- Will always find a page or loop forever?
  - Even if all use bits set, will eventually loop all the way around FIFO
- What if hand moving slowly?
  - Good sign or bad sign?
    - » Not many page faults
    - » or find page quickly
- What if hand is moving quickly?
  - Lots of page faults and/or lots of reference bits set



This provides a brilliant, low-overhead approximation of LRU that powers the operating systems today

Clock Algorithm problem

Consider the following page reference string:

1, 2, 3, 2, 4, 1

- Assume that the system has **3-page frames**. Apply the **Clock Page Replacement Algorithm** to the given reference string.

Tasks:

Show the contents of the page frames after each page reference.

Indicate whether each page reference results in a **Page Hit** or a **Page Fault**.

Determine the **total number of page faults**.

Draw the final state of the clock (circular queue) showing the page frames and the clock pointer.



THANK YOU

Pavan A C
Neha Sharma
Department of Computer Science and Engineering